

Software Architecture and Design Specification

Project: Blood Bank Management System (BBMS)
Version: 1.0
Authors: Team T9 - Uttam (PES1UG24CS697); Abhinav K (PES1UG24CS701); Akshay Arcot (PES1UG24CS705); Navaneeth Tanuboddi (PES1UG24CS709)
Date: 27-09-2026
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	27-09-2026	Team T9	Initial Software Architecture & Design Specification.

Approvals

Role	Name	Signature/Date
Project Lead		
Faculty / Course Evaluator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Blood Bank Management System (BBMS). It translates the functional, non-functional, and security requirements of the BBMS SRS into an implementable web-application architecture: component decomposition, interfaces, interaction flows, API contract, error handling, and security architecture.

1.2 Scope

Covers donor, hospital, blood bank staff, and administrator workflows; blood collection and unit identification; serology and quarantine; component separation; inventory and FEFO; requisitions and STAT handling; ABO/Rh compatibility and allocation; dispatch and delivery; audit logging; reporting; and public availability. Excludes national/regional registry integration, payment processing, direct laboratory-equipment integration, native mobile applications, and multi-language localization (SRS 1.2).

1.3 Audience

Developers (Team T9), QA engineers, security auditors, course faculty/evaluators, and future maintainers of the academic prototype.

1.4 Definitions

BBMS, SRS, STP, RTM, RBAC, FEFO, API, REST, ORM, ABO/Rh, STAT, Bag ID, ADR, STRIDE, TLS. Full definitions are in Appendix 5.1.

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture (goals, stakeholders, UML component diagram, components, pattern rationale and ADRs, technology stack, risks, traceability, and the STRIDE security architecture). Section 4 gives the design (modules and state rules, two UML sequence diagrams, API design, error handling/logging/monitoring, UX design, and open issues). Section 5 holds the glossary, references, and tools.

2.2 Related Documents

BBMS SRS v1.0 (requirements and RTM); BBMS Software Test Plan BBMS-TP-001 v1.0 (STP); Requirements Traceability Matrix (SRS Section 8 and STP Section 13).

3. Architecture

3.1 Goals & Constraints

Goals:

- Separate presentation, API/control, business logic, data access, and persistence responsibilities.
- Enforce server-side authentication and RBAC for every protected function.
- Protect blood-unit safety state transitions and maintain auditability of data-changing actions.
- Support the measurable SRS quality requirements: performance, reliability, accessibility, portability, maintainability, and future scalability.
- Stay compatible with either the Django/Python or Node.js/Express backend option in the SRS.

Constraints and assumptions:

- SQLite is an embedded database; expected concurrency is limited to the scale of a university mini-project.
- The server is authoritative for timestamps, expiry calculations, and audit events.
- Serology is performed outside BBMS; staff enter Pass/Fail values manually.
- SMTP is an external dependency used for optional email notifications.
- The application is a prototype and is not certified for clinical or regulatory use.

Quality attribute drivers:

Quality attribute	Design response	SRS references
Performance	Indexed queries, lightweight REST endpoints, ORM access, FEFO query ordering	NF-001–NF-003
Reliability	Transactional persistence, restart-safe committed data, controlled status transitions	NF-004–NF-006
Usability / Accessibility	Role-specific dashboards, field-level validation, WCAG 2.1 AA target	NF-007–NF-009
Maintainability	Layered modules and service boundaries; automated test target	NF-010–NF-011
Portability	Browser-based client; backend isolated behind framework-neutral interfaces	NF-012–NF-013
Scalability	Location-aware inventory model and data-access abstraction	NF-014–NF-015
Security	Authentication, RBAC, validation, secure cookies, rate limits, audit logging	SR-001–SR-008

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Donors	Privacy of personal and medical data; simple registration, eligibility, and booking.
Hospital users	Timely and correct allocation; STAT prioritization; status tracking and delivery confirmation.
Blood bank staff	Unit safety (no unsafe or expired unit dispatched); FEFO selection; clear quarantine and inventory views.
Administrators	Accountability through audit logs; user, role, and hospital-approval management; reports.
Public / guests	Current blood availability without exposure of donor or hospital identities.
Course evaluators / security auditors	Traceability to the SRS and STP; verifiable security controls.
Developers / maintainers	Modularity, framework-neutral contracts, clear boundaries for future RDBMS or multi-location extension.

3.3 Component (UML) Diagram

System context: External users interact with BBMS through a modern web browser. The frontend exchanges JSON with the REST API over HTTPS. The API delegates authenticated work to domain services and persists data through an ORM or equivalent data-access layer into SQLite. Notifications are sent through SMTP. The public availability page uses only sanitized stock data and does not expose donor- or hospital-identifying information.

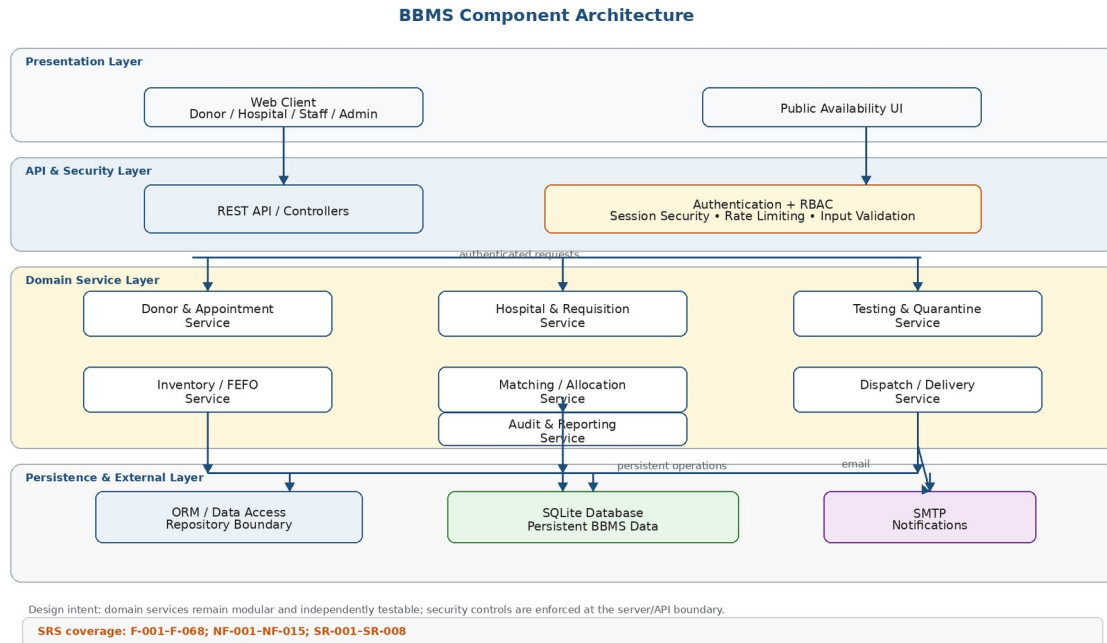


Figure 3.1 — BBMS Component Architecture

The diagram shows the main application responsibilities and their key interactions. Authentication and RBAC are cross-cutting controls rather than business-domain modules; Audit & Reporting is similarly cross-cutting for accountability and operational reporting.

3.4 Component Descriptions

Component	Responsibility	Primary SRS coverage
Web Client	Role-specific dashboards, forms, validation feedback, public availability UI	F-001–F-068; NF-007–NF-009
REST API Layer	HTTP routing, request/response mapping, validation entry points, controller logic	F-001–F-068; SR-001–SR-008
Authentication & RBAC	Credential verification, session lifecycle, role authorization, logout/rate limiting	F-001–F-005; F-040–F-042; F-060–F-063; SR-001–SR-007
Donor Management	Profiles, eligibility and appointment workflows	F-006–F-019
Hospital & Requisition	Hospital onboarding, requisitions, STAT priority, tracking	F-040–F-048
Testing & Quarantine	Serology results, quarantine/discard rules, component separation	F-024–F-032
Inventory / FEFO	Units, counts, status, expiry, FEFO selection, location inventory	F-020–F-039
Compatibility & Allocation	ABO/Rh matching, cross-match outcomes, reservations	F-049–F-052
Dispatch & Delivery	Dispatch records, custody confirmation, status transitions	F-053–F-057
Audit & Reporting	Immutable/tamper-evident events, operational reports, CSV export	F-058–F-066; SR-008
Public Availability	Unauthenticated, privacy-preserving stock view	F-067–F-068

Component	Responsibility	Primary SRS coverage
SQLite via ORM	Persistent storage and transaction boundary	NF-004–NF-006; SR-004

Component interaction:

- Donor booking calls Eligibility before the Appointment Service creates the appointment.
- Blood collection creates a Blood Unit in Quarantined state and updates donor history.
- Serology/Quarantine controls transition units to Available or Discarded and feed Inventory.
- Hospital requests are validated, prioritized, and passed to Matching/Allocation for compatible FEFO candidates.
- Dispatch records change allocated units to Dispatched; hospital receipt changes them to Delivered.
- All data-changing and security-relevant actions emit audit events.

Data architecture: The logical data model is derived from the SRS domain objects. Core entities: User/Role, Donor, Hospital, Appointment, Donation/Collection, BloodUnit, SerologyResult, BloodComponent, Inventory/Location, Requisition, Allocation/CrossMatch, Dispatch, DeliveryConfirmation, AuditLog, and reportable aggregates. Foreign-key relationships preserve ownership and traceability; separated component records stay linked to their originating Bag ID, and requisition → allocation → dispatch history is kept intact.

3.5 Chosen Architecture Pattern and Rationale

A layered client-server architecture with an MVC-inspired web/API implementation was chosen. The logical layers are (1) Presentation/Web Client, (2) REST API / Controller layer, (3) Application Services, (4) Persistence / ORM, and (5) SQLite storage. Django or Express can implement the web/API layer while the service and persistence boundaries stay stable, which supports NF-010 and keeps changes localized by module. Microservices were rejected as overly complex for a single-host mini-project on an embedded database. The key decisions are recorded as ADRs:

ADR	Decision	Rationale
ADR-01	Layered client-server architecture isolating UI, authorization, domain services, and persistence.	Clear separation of concerns; localized change (NF-010). Microservices rejected: excessive for this scale.
ADR-02	Treat all security controls as server-side responsibilities; client-side hiding is never sufficient.	Client controls can be bypassed; server checks are authoritative (SR-003).
ADR-03	Explicit blood-unit and requisition state transitions with guards.	Prevents unsafe or expired units from being allocated or dispatched.
ADR-04	Keep API contracts stable across the permitted Django or Node.js backends.	Avoids framework divergence; supports portability (NF-013).
ADR-05	Use SQLite as specified, with a clean ORM/data-access boundary.	Meets mini-project scope; allows future RDBMS migration (NF-014).
ADR-06	Single centralized, append-oriented audit pipeline for business and security events.	Complete accountability (F-058, SR-008); one path for the STP to validate.

3.6 Technology Stack & Data Stores

Layer	Technology
Client	HTML, CSS, JavaScript in a modern browser (latest two stable Chrome, Firefox, Edge)
API / application	Python 3.10+ with Django (or Django REST Framework), or Node.js 18+ with Express; RESTful JSON API
Data access	ORM / parameterized data-access layer
Data store	SQLite embedded database (persistent file; WAL mode per deployment view)
Notifications	SMTP email service (optional confirmations and STAT alerts)
Transport / security	HTTPS/TLS; HttpOnly/Secure session cookies; salted one-way password hashes

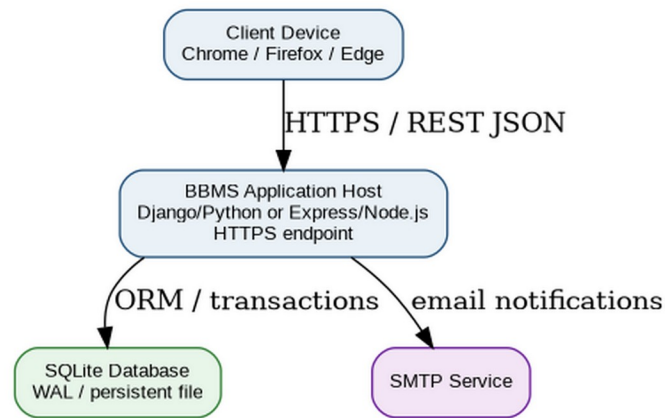


Figure 3.2 — Logical Deployment Architecture

3.7 Risks & Mitigations

Risk	Impact	Mitigation
SQLite concurrent-write limits	Reduced throughput during load	Keep deployment within SRS scope; measure against NF-001–NF-003.
Incorrect clinical configuration	Unsafe business decision	Make eligibility disqualifiers, expiry rules, and compatibility data configurable and test-driven.
SMTP unavailable	Email tests blocked	Keep on-screen confirmations as primary UI behavior; mark dependent email tests accordingly.
Framework divergence (Django vs Node)	Inconsistent implementation	Maintain logical component and API contracts independent of framework.
Incomplete audit coverage	Reduced accountability	Use a centralized audit service invoked from all data-changing/security paths.

Limitations: the design does not cover laboratory-equipment or national-registry integration (outside SRS scope); SQLite and single-host deployment suit the academic mini-project but not a production multi-hospital model; clinical policy values (eligibility conditions, shelf lives, compatibility rules) are configurable system rules, and no regulatory certification is claimed.

3.8 Traceability to Requirements

Architecture element – SRS requirements:

Architecture element	Functional	Non-functional	Security
Web / API	F-001–F-019, F-040–F-048, F-060–F-068	NF-007–NF-009	SR-001, SR-003, SR-005, SR-006
Auth / RBAC	F-001–F-005, F-040–F-042, F-060–F-063	NF-005	SR-001–SR-003, SR-006–SR-007
Collection / Unit	F-017–F-023	NF-004–NF-006	SR-004, SR-008
Testing / Quarantine	F-024–F-032	NF-004, NF-005	SR-004, SR-008
Inventory / FEFO	F-033–F-039	NF-001–NF-003, NF-014–NF-015	SR-004, SR-008
Hospital / Requisition	F-040–F-048	NF-001–NF-003	SR-001, SR-003, SR-005, SR-008
Matching / Allocation	F-049–F-052	NF-001–NF-003	SR-004, SR-008
Dispatch / Delivery	F-053–F-057	NF-004–NF-006	SR-003, SR-008
Audit / Reporting	F-058–F-066	NF-006, NF-010–NF-011	SR-008
Public Availability	F-067–F-068	NF-001–NF-009	SR-005, SR-008

Design element – STP test cases:

Design element	Primary test cases
Authentication / session	TC-AUTH-01, TC-AUTH-02, TC-SEC-02, TC-SEC-06, TC-SEC-07
Donor eligibility / appointment	TC-DNR-01, TC-ELG-01, TC-APT-01, TC-APT-02
Blood collection / Bag ID	TC-COL-01, TC-BAG-01, TC-BAG-02
Serology / quarantine / components	TC-SER-01, TC-SER-02, TC-QNT-01, TC-CMP-01
Inventory / FEFO / expiry	TC-INV-01, TC-INV-02, TC-EXP-01
Hospital requisition / STAT	TC-AUTH-03, TC-REQ-01, TC-REQ-02, TC-REQ-03
Compatibility / allocation	TC-MATCH-01
Dispatch / delivery	TC-DISP-01, TC-DEL-01
Audit / admin / reports / public portal	TC-AUD-01, TC-ADM-01, TC-RPT-01, TC-PUB-01
Performance / reliability / accessibility / portability	TC-NF-01 to TC-NF-06
Security architecture	TC-SEC-01 to TC-SEC-08

Test statuses remain “Not Run” until execution; see the STP for records.

3.9 Security Architecture

Security objectives:

- Protect donor personal and medical information from unauthorized access or disclosure.
- Ensure only authenticated and authorized users perform role-appropriate operations.
- Preserve blood-unit testing and status integrity so unsafe blood cannot be allocated or dispatched.

- Maintain accountability through security and data-change audit records.

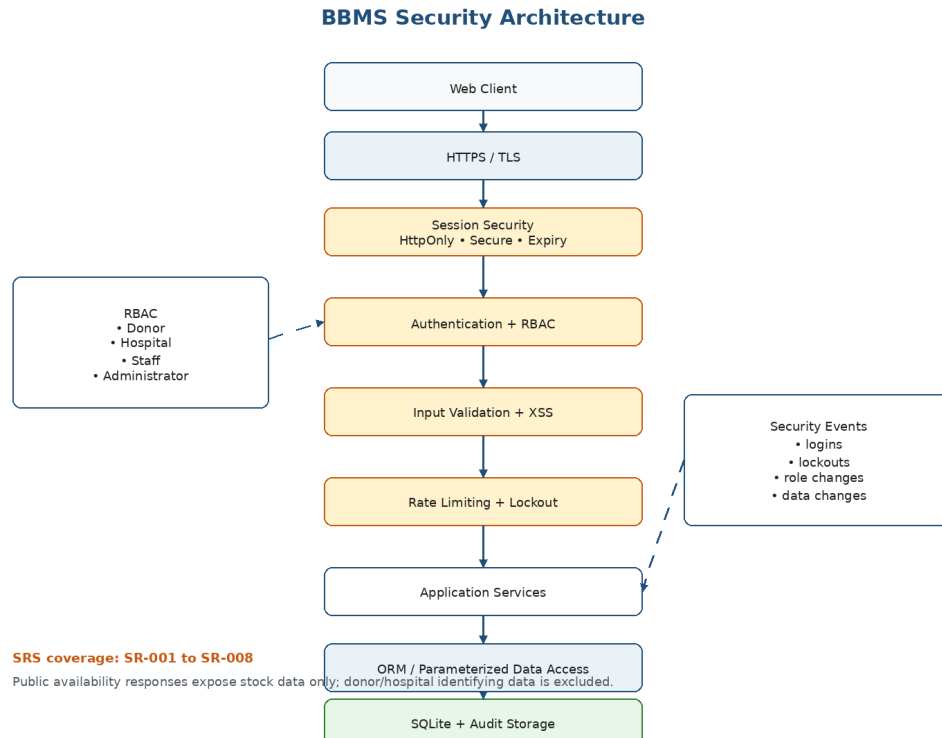


Figure 3.3 — BBMS Security Architecture

Security is enforced at multiple layers: encrypted transport, session protection, server-side authentication and RBAC, input validation and output encoding, abuse controls, ORM/parameterized data access, and audit logging. Client-side controls are usability aids only.

Security controls:

- **Authentication and sessions (SR-001, SR-002, SR-006, SR-008):** every role authenticates before role-specific access; passwords stored as salted one-way hashes and never logged or returned; session tokens in HttpOnly and Secure cookies with defined expiry, invalidated on logout and after 30 minutes of inactivity; HTTPS/TLS for all client-server traffic.
- **Authorization / RBAC (SR-003):** every protected endpoint is checked server-side against user identity, role, and account state. Donors cannot reach staff/admin functions, unapproved hospitals cannot submit requisitions, and only administrators reach account and audit management. Unauthorized requests return HTTP 403.
- **Input and data protection (SR-004, SR-005, F-068):** validate types, ranges, required fields, and business constraints before service execution; ORM or parameterized queries only; encode browser-rendered output; exclude donor- and hospital-identifying data from the public availability response.
- **Rate limiting and abuse protection (SR-007):** at most 10 login requests per minute per source IP; account locked for 15 minutes after five consecutive failures; applied before expensive authentication work and logged as security events.

- **Audit and accountability (F-058, F-059, SR-008):** events for creations, status changes, and deletions of donors, blood units, requisitions, and user accounts (actor, action, timestamp, record ID), plus login attempts, lockouts, and role changes; the full log is administrator-only and read-only/non-deletable.

Threat modeling (STRIDE):

Threat	Example in BBMS	Mitigation	Ref.
Spoofing	Attacker impersonates a donor, staff member, or hospital; credential guessing	Authenticated sessions, salted password hashes, 15-minute lockout after 5 failures, per-IP rate limit	SR-001, SR-002, SR-007
Tampering	Modified requests, SQL injection, altered unit status, edited audit records	ORM/parameterized queries, server-side validation, guarded state transitions, HTTPS/TLS, append-only audit records	SR-004, SR-005, SR-008
Repudiation	A user denies an allocation, dispatch, or role change	Audit trail with actor, action, timestamp, record ID; server-authoritative timestamps	F-058, SR-008
Information disclosure	Donor medical data leaked; session theft; identifiers on public page; stack traces in errors	TLS; HttpOnly/Secure cookies; sanitized public endpoint; generic client-safe errors; no secrets in logs	SR-002, SR-006, SR-008, F-068
Denial of service	Login flooding; heavy availability queries; SQLite write contention	Per-IP rate limiting, lockout, indexed queries, load kept within SRS scope	SR-007, NF-001–NF-003
Elevation of privilege	Donor or hospital calls staff/admin endpoints; unapproved hospital submits requisitions	Server-side RBAC middleware returning HTTP 403; hospital approval gate; admin-only audit and account management	SR-003, F-040–F-042

Security traceability:

SRS requirement	Architecture control	STP coverage
SR-001	Authentication service and protected API middleware	TC-SEC-01
SR-002	Password hashing in authentication service; credential secrecy	TC-SEC-02
SR-003	Server-side RBAC middleware; HTTP 403 on unauthorized access	TC-SEC-03
SR-004	ORM / parameterized data-access layer	TC-SEC-04
SR-005	Schema validation + output encoding / XSS defenses	TC-SEC-05
SR-006	HttpOnly/Secure cookies, expiry, logout, inactivity timeout	TC-SEC-06
SR-007	Login lockout + per-IP rate limiting	TC-SEC-07
SR-008	HTTPS/TLS + security-event / data-change audit trail	TC-SEC-08, TC-AUD-01

4. Design

4.1 Design Overview

BBMS is designed as layered, single-responsibility modules that remain separate even when deployed as one process. Principles: explicit state transitions (unsafe or expired units are never available by default); authoritative server-side validation and authorization; traceability

(records keep the IDs and timestamps needed to follow donor → collection → unit → requisition → dispatch → delivery); and stable API contracts independent of the backend framework.

Module design:

Module	Key operations	Important rules / SRS
Donor	Register, authenticate, profile, questionnaire, appointment	18+ registration; unique email; eligibility before booking; 90-day default interval; F-001–F-016
Collection / Blood Unit	Record collection, volume/time, create Bag ID, blood group	Unique Bag ID; lab-confirmed blood group distinct from self-report; F-017–F-023
Serology / Quarantine	Record Pass/Fail, status transitions, discard	Default Quarantined; all required tests Pass before Available; any Fail → Discarded; F-024–F-030
Components / Inventory	Separate units, inventory counts, expiry, FEFO	Trace components to source Bag ID; exclude unsafe/expired; F-031–F-039
Hospital / Requisition	Register, approve hospital, create requests, STAT	Unapproved hospitals cannot submit; unique Pending request; F-040–F-048
Matching / Allocation	Compatibility list, cross-match, reserve units	Compatible groups only unless explicit authorized override; F-049–F-052
Dispatch / Delivery	Create dispatch, update unit/request, confirm receipt	Record staff/timestamp; Dispatched then Delivered; F-053–F-057
Audit / Reports	Audit records, admin view, CSV reports	Read-only audit display; activity and inventory reports; F-058–F-066
Public Availability	Show stock summary	Unauthenticated; no identifying data; F-067–F-068

State management: a collected blood unit starts as Quarantined. Only when all required serology results are Pass may it become Available; any Fail makes it Discarded and non-allocatable. Allocated and dispatched units become Dispatched, and Delivered after hospital confirmation. Expired units are excluded from availability and allocation.

Object	State transition	Guard / condition
Blood Unit	Collected → Quarantined	Created at collection
Blood Unit	Quarantined → Available	All required serology results = Pass
Blood Unit	Quarantined/Available → Discarded	Any serology result = Fail / safe-disposal action
Blood Unit	Available → Expired	Expiry date passed
Blood Unit	Available → Dispatched	Valid allocation + dispatch
Blood Unit	Dispatched → Delivered	Hospital receipt confirmation
Requisition	New → Pending	Successful submission
Requisition	Pending → Approved / Partially Fulfilled / Rejected	Staff action
Requisition	Approved → Fulfilled / Partially Fulfilled	Dispatch quantity determines final status

4.2 UML Sequence Diagrams

Flow 1 — Donor registration, eligibility, and appointment booking. Authentication establishes the donor session first; the eligibility service checks configured conditions and the donation interval; the appointment service then persists the slot reservation and optional notification. This enforces the SRS rule that eligibility screening precedes appointment confirmation.

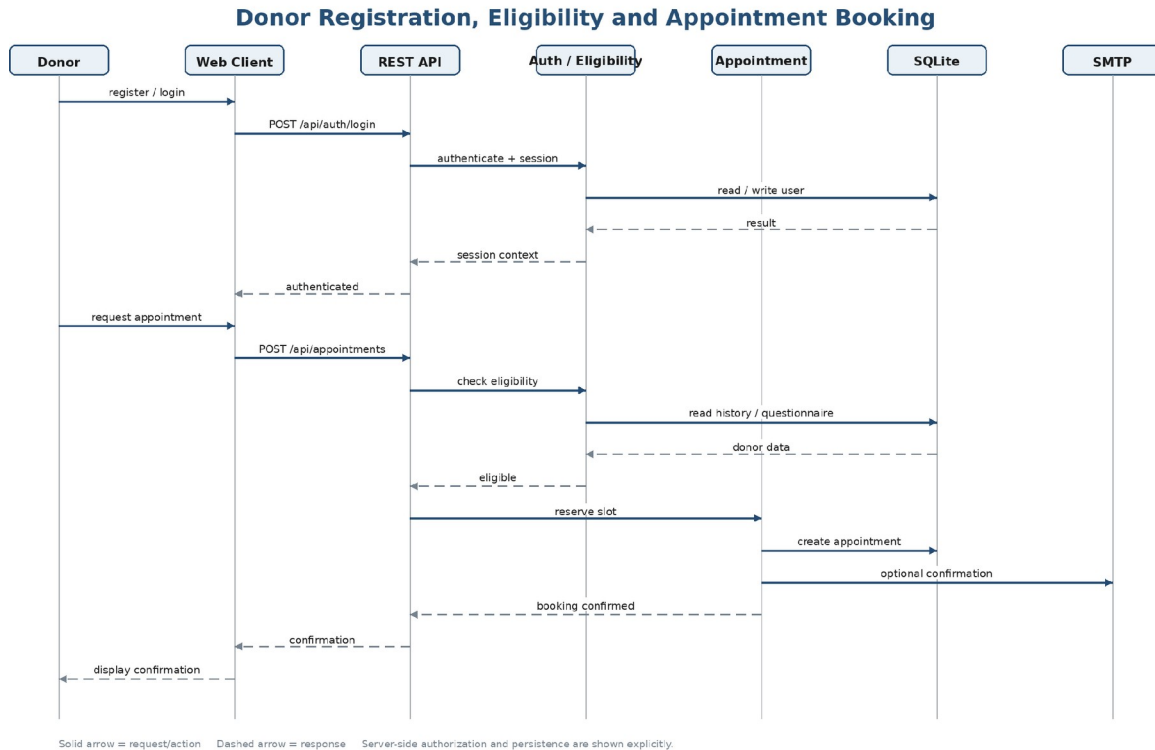


Figure 4.1 — Donor Registration, Eligibility, and Appointment Booking Sequence

Flow 2 — Hospital requisition, allocation, dispatch, and delivery confirmation. The flow shows server-side hospital approval checking, Pending request creation, STAT prioritization, staff approval, compatibility/FEFO selection, reservation, dispatch recording, and delivery confirmation. Each committed state transition is persistently recorded and emits an audit event.

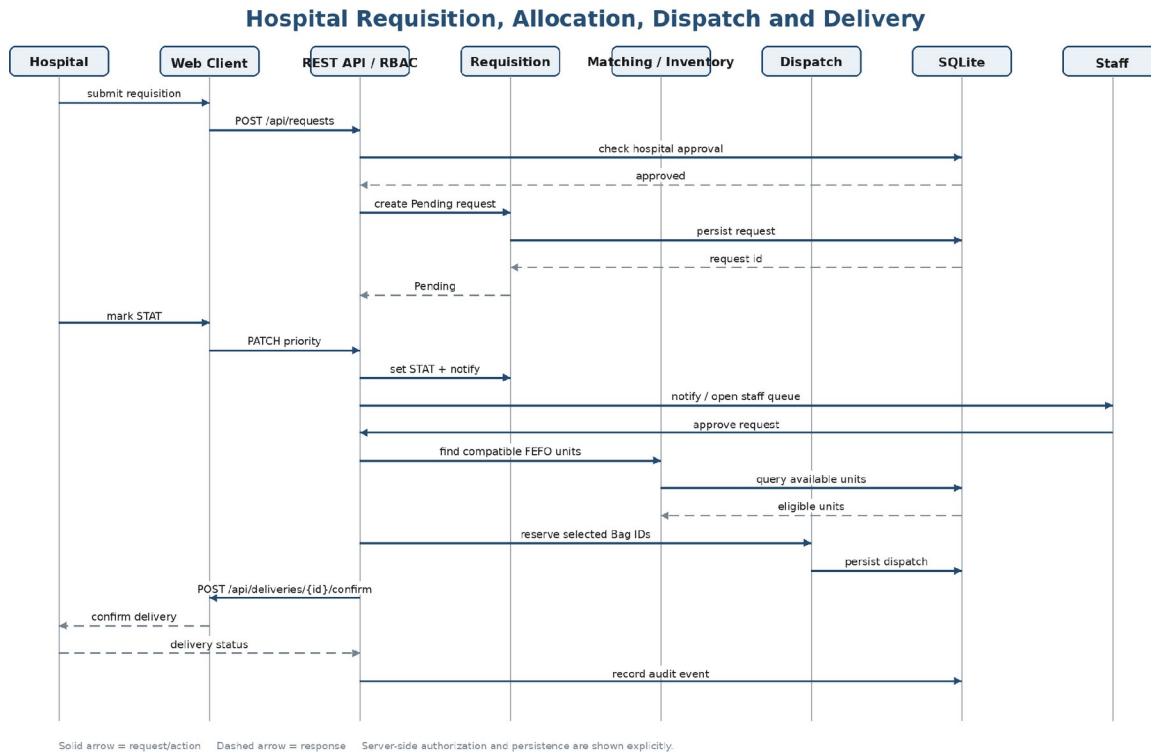


Figure 4.2 — Hospital Requisition, Allocation, Dispatch, and Delivery Confirmation Sequence

4.3 API Design

The SRS requires a RESTful JSON API over HTTPS. The table is the design contract for the major workflows; implementation may use Django REST Framework, Django views, Express routes, or equivalent.

Method	Endpoint	Role	Purpose	SRS
POST	/api/auth/register	Public	Create donor account	F-001–F-003
POST	/api/auth/login	Public	Authenticate user / create session	F-004; SR-001, SR-006, SR-007
POST	/api/auth/password-reset/request	Public	Request password reset	F-005
POST	/api/auth/logout	Authenticated	Invalidate current session	SR-006
GET/PUT	/api/donors/me	Donor	View/update donor profile	F-006
GET	/api/donors/me/donations	Donor	View donation history	F-007
GET	/api/donors/me/eligibility	Donor	Next eligible date / status	F-008–F-011
POST	/api/appointments	Donor	Book eligible donor into slot	F-012–F-016
PATCH	/api/appointments/{id}	Donor	Reschedule/cancel appointment	F-014–F-015
POST	/api/collections	Staff	Record collection and create Bag ID	F-017–F-021
POST	/api/blood-units/{bagId}/serology	Staff	Record required test results	F-024–F-027
GET	/api/inventory	Staff	Filter inventory / availability	F-028, F-033–F-039
POST	/api/blood-units/{bagId}/components	Staff	Record component separation	F-031–F-032

Method	Endpoint	Role	Purpose	SRS
POST	/api/hospitals/register	Public / Hospital	Submit hospital registration	F-040
POST	/api/requests	Hospital	Submit blood requisition	F-043–F-046
PATCH	/api/requests/{id}/priority	Hospital	Mark request Emergency/STAT	F-047–F-048
POST	/api/requests/{id}/cross-match	Staff	Compute compatibility / record result	F-049–F-051
POST	/api/requests/{id}/allocate	Staff	Reserve specific Bag IDs	F-052
POST	/api/dispatch	Staff	Create dispatch and manifest	F-053–F-055
POST	/api/deliveries/{id}/confirm	Hospital	Confirm receipt / custody	F-056–F-057
GET	/api/audit	Admin	Read-only audit log	F-058–F-059
GET	/api/reports/inventory.csv	Admin	Export inventory report	F-064
GET	/api/reports/donations.csv	Admin	Export donation activity report	F-065
GET	/api/reports/requisitions.csv	Admin	Export requisition fulfilment report	F-066
GET	/api/public/availability	Public	Read-only stock availability	F-067–F-068

Interface definitions (representative components). Field names below are the design contract; the SRS/implementation may add optional fields.

Item	Authentication service — login	Requisition service — submit request	Dispatch service — create dispatch
Endpoint	/api/auth/login	/api/requests	/api/dispatch
Method / role	POST / Public	POST / Hospital (approved)	POST / Staff
Request	{email, password}	{bloodGroup, component, quantity, requiredBy}	{requestId, bagIds[]}
Response	200 {user:{id, role}}; session set via HttpOnly, Secure cookie	201 {requestId, status:"Pending"}	201 {dispatchId, requestId, bagIds[], dispatchedBy, dispatchedAt, requestStatus}
Errors	400 invalid fields; 401 generic authentication failure; 429 rate limit / account locked	400 invalid fields; 401 not authenticated; 403 not a hospital or hospital not approved	400 invalid fields; 401; 403 not staff; 404 request/unit not found; 409 unit not Available, not allocated, or incompatible
SRS / STP	F-004, SR-001, SR-006, SR-007 / TC-AUTH-01, TC-SEC-07	F-043–F-046 / TC-REQ-01	F-053–F-055 / TC-DISP-01

API request pipeline:

Stage	Behavior	Security ref.
1. Transport	Require HTTPS/TLS.	SR-008
2. Session	Read authenticated session from secure cookie; reject absent/expired session on protected routes.	SR-001, SR-006
3. Role check	Authorize endpoint against the role-permission matrix.	SR-003
4. Input validation	Validate request schema and business constraints.	SR-005

Stage	Behavior	Security ref.
5. Service call	Execute domain operation using trusted service context.	F-001–F-068
6. Audit	Record required security/data-change event.	F-058, SR-008

4.4 Error Handling, Logging & Monitoring

Error handling:

HTTP	Meaning	Response / action
400	Invalid or malformed input	Return field-level validation errors; do not execute the domain operation.
401	Missing or invalid authentication	Return generic authentication failure; do not disclose credential details.
403	Authenticated but not authorized	Return forbidden response; no protected resource is exposed.
404	Resource not found	Return stable not-found error; do not reveal unrelated identifiers.
409	Business conflict	Examples: duplicate email/Bag ID, full slot, incompatible status transition.
429	Rate limit exceeded	Return retry guidance and enforce login/request throttle.
500	Unexpected application error	Log server-side with timestamp/stack trace; return a generic client-safe message (NF-006).

Standard error envelope: {"error":{"code":"VALIDATION_ERROR","message":"Request contains invalid fields","fields":{"email":"Invalid email format"}}}. Error details returned to clients never contain passwords, stack traces, database credentials, or other secrets.

Logging:

- **Audit log:** append-oriented record model. Each event stores actor identity, action, timestamp, affected record ID, and enough context to reconstruct the change. No edit/delete operations are exposed; administrator UI access is read-only. Business and security events share one pipeline so the STP can validate both.
- **Application error log:** unhandled errors recorded server-side with timestamp and stack trace (NF-006).
- **Log hygiene:** plaintext passwords, session tokens, and secrets are never written to logs (SR-002).

Monitoring (prototype level, derived from log and report data):

- Availability of the application against the 99% scheduled-availability target (NF-004).
- Response time of blood-availability search and appointment booking against NF-001 and NF-002.
- Rate of HTTP 5xx errors and unhandled exceptions.
- Failed-login, lockout, and rate-limit events recorded as security events.
- SMTP delivery failures for notification emails.

4.5 UX Design

- Role-specific dashboards for donors, hospital users, staff, and administrators; a public availability page requiring no login.

- Field-level validation messages that identify the affected field and the issue (NF-009).
- Accessibility targeting WCAG 2.1 AA: sufficient colour contrast and full keyboard navigation for primary workflows (NF-008).
- A first-time donor can complete registration unaided in under 5 minutes (NF-007).
- Responsive web layout for common screen sizes; no native mobile application (SRS 1.2).
- On-screen confirmations are the primary feedback; email is optional. STAT requests raise an on-screen staff alert (F-048).
- English only; multi-language localization is out of scope (SRS 1.2).

4.6 Open Issues & Next Steps

- Final backend choice (Django vs Node.js/Express); the API contract in 4.3 is framework-neutral.
- Confirm clinical configuration values (eligibility disqualifiers, component shelf lives, compatibility rules) with the course-approved sources.
- Set up the HTTPS/TLS test environment and an SMTP service or test mail sink.
- Measure SQLite behavior against NF-001–NF-003 and record observed limits.
- Execute the STP (all statuses currently Not Run) and update the RTM.
- Future enhancements: migration to a production RDBMS and additional blood-bank locations (NF-014, NF-015).

4.7 Conclusion

This specification translates the BBMS SRS into a coherent layered architecture and detailed design. The component structure separates the donor, hospital, inventory, testing, matching, dispatch, audit, and security responsibilities; the sequence diagrams show the principal end-to-end workflows; the REST API design defines the system interaction boundary; and the security architecture addresses authentication, RBAC, input protection, session security, rate limiting, TLS, and auditability. The traceability sections align the architecture and design with the SRS requirement identifiers and the companion Test Plan test cases.

5. Appendices

5.1 Glossary

Term	Meaning
BBMS	Blood Bank Management System
SRS / STP / SAD	Software Requirements Specification / Software Test Plan / Software Architecture & Design Specification
RTM	Requirements Traceability Matrix
RBAC	Role-Based Access Control
FEFO	First-Expiry-First-Out
API / REST	Application Programming Interface / Representational State Transfer
ORM	Object-Relational Mapping

Term	Meaning
ABO/Rh	Blood group compatibility classification
STAT	Emergency / immediate-priority requisition
Bag ID	Unique identifier of a collected blood unit
ADR	Architecture Decision Record
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS / HTTPS	Transport Layer Security / HTTP over TLS
SMTP	Simple Mail Transfer Protocol
WCAG	Web Content Accessibility Guidelines

5.2 References

- BBMS Software Requirements Specification v1.0.
- BBMS Software Test Plan (BBMS-TP-001) v1.0.
- IEEE Std 830-1998 (structural reference used by the SRS).
- STRIDE threat-modeling method (Section 3.9).

5.3 Tools

- Django / Django REST Framework or Node.js / Express; SQLite.
- Supertest and the Node.js test runner; ApacheBench; Lighthouse (per STP Section 6).
- Chrome, Firefox, and Edge for portability testing.